rocket.chat

Google Summer Of Code 2026 - Proposal

New Users Anti Spammer System

About Me

Name: **Meet Jain**

Rocket.Chat: meet.jainn

Email: mjmeet.279@gmail.com

GitHub: not-meet

Linkedin: meet-jain

Alfredo Del Fabro Neto,Gabriel Casals

Rocket.Chat Mentors

Synopsis

Open community workspaces on Rocket.Chat are increasingly vulnerable to coordinated spam where new accounts join, flood multiple public channels with identical or slightly varied messages, and vanish before any moderator even notices. The New Users Anti-Spam Monitoring System shifts this dynamic from reactive to proactive by monitoring new users through their first 6 to 10 weeks monitoring window, combining smart detection with a graduated human validated restriction system. Detection catches duplicate and polymorphic spam across channels while the restriction system levels ensures the response is always proportional, escalating naturally with repeat behavior and never taking permanent action without an admin making the final call. Every flag is explainable, every action is reversible, and moderators stay in full control throughout.

Project overview

Rocket.Chat powers thousands of open communities where anyone can join and start participating almost instantly. That openness is one of its greatest strengths, but it also creates a real vulnerability. Spam campaigns targeting these workspaces follow a predictable pattern where a new account joins, immediately starts posting the same or slightly varied message across multiple public channels, and causes genuine disruption before any moderator has a chance to respond.

The New Users Anti-Spam Monitoring System is built to change that. It is a Rocket.Chat App built on the Apps Engine that hooks directly into the message pipeline and monitors new users through their first 6 to 10 weeks monitoring window. The detection layer runs multiple gates in sequence, catching exact duplicates through hash matching, polymorphic variations through cosine similarity, and cross channel floods by tracking how the same message spreads across rooms within a sliding time window, all fast enough to act before a message even reaches the database.

What makes this system different from a simple spam filter is the Moderation Level system, a four tier graduated response that ensures every action is proportional to the behavior. A first offence gets a warning, repeat behavior escalates through cooldowns, and a permanent block only happens when an admin reviews and decides. Nothing is irreversible, nothing is a black box, and the admin dashboard ensures full visibility into every flagged user and every action taken.

Benefits to the community

This project will bring meaningful improvements to how Rocket.Chat communities handle trust and safety, benefiting admins, moderators, and everyday users alike.

For Community Members:

Public channels will be significantly cleaner and safer as spam is caught and contained before it spreads across rooms. Legitimate users get a better first experience and genuine conversations are no longer buried under coordinated spam floods.

For Admins and Moderators:

The shift from reactive to proactive moderation means admins spend less time firefighting and more time actually managing their communities. The Moderation Level system ensures every response is proportional and explainable, the admin dashboard gives complete visibility into flagged users and their history, and no permanent action ever happens without a human making the final call.

For the Broader Platform:

A configurable anti-spam application makes Rocket.Chat meaningfully more viable for large open communities that currently struggle with moderation at scale. The system is designed to be transparent, reversible, and admin friendly, setting a strong foundation for trust and safety tooling that the platform can build on going forward.

Deliverables

By the end of the Google Summer of Code period, I plan to deliver the following with the New Users Anti-Spam Monitoring System:

- **Fully Functional Rocket.Chat Anti-Spam App:** A complete, installable Rocket.Chat App built on the Apps Engine that any workspace admin can set up and run, covering the full detection pipeline, Moderation Level system, and admin dashboard out of the box.

- **Configurable Moderation Level System:** A fully customizable four tier restriction system where admins can define thresholds, cooldown durations, and escalation behavior directly from the admin panel without touching any code.

- **Behavioral Scoring Engine:** A complete implementation of the deeper behavioral analysis layer that tracks posting frequency, link density, engagement diversity, and other signals across the full monitoring window to produce a meaningful risk score per new user.

- **Daily Risk Scoring and Reporting:** An automated daily report delivered to the admin channel summarizing new user activity, current risk scores, flagged users, and any actions taken, giving admins a clear picture of workspace health without having to actively check.

- **Enriched Admin Notifications:** Flagged user alerts that include a curated message history and behavioral context so admins have everything they need to make an informed decision when vouching or confirming a restriction.

- **Admin Configuration Panel:** A clean settings interface where workspace admins can tune detection sensitivity, monitoring window duration, Moderation Level thresholds, and notification preferences to match the needs of their specific community.

- **Test Coverage and Documentation:** Thorough testing across a range of spam patterns including exact duplicates, polymorphic variations, and cross channel floods, alongside clear documentation covering architecture decisions, known limitations, and configuration guidance for future maintainers.

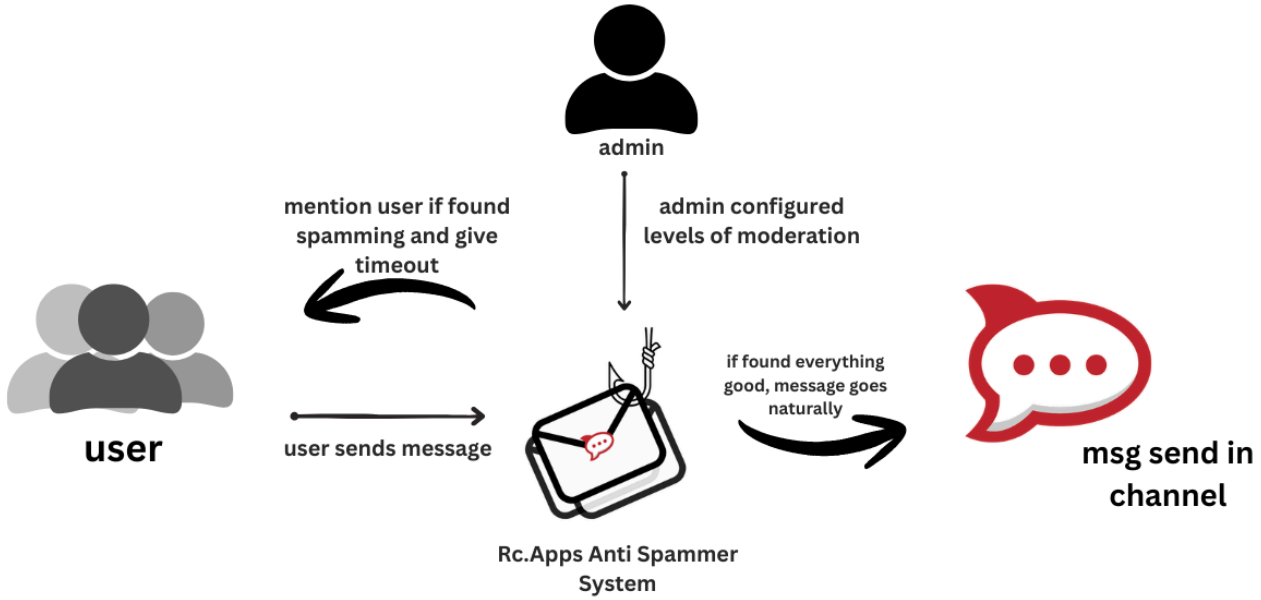
Future Scopes

After the Google Summer of Code period concludes, I intend to continue actively developing and refining the app with the guidance of my mentors until it reaches a stable release. The following are additional directions I plan to explore:

- **Reputation Portability:** Build a mechanism where a user's trust record can carry over if they rejoin under the same identity, preventing the common pattern of ban evasion through account recreation while keeping the process fair and admin reviewed.

- **Natural Language Admin Commands:** Allow admins to interact with the system through simple slash commands or natural language inputs directly in the admin channel, making it faster to query flagged users, adjust thresholds, or review recent activity without opening the dashboard.

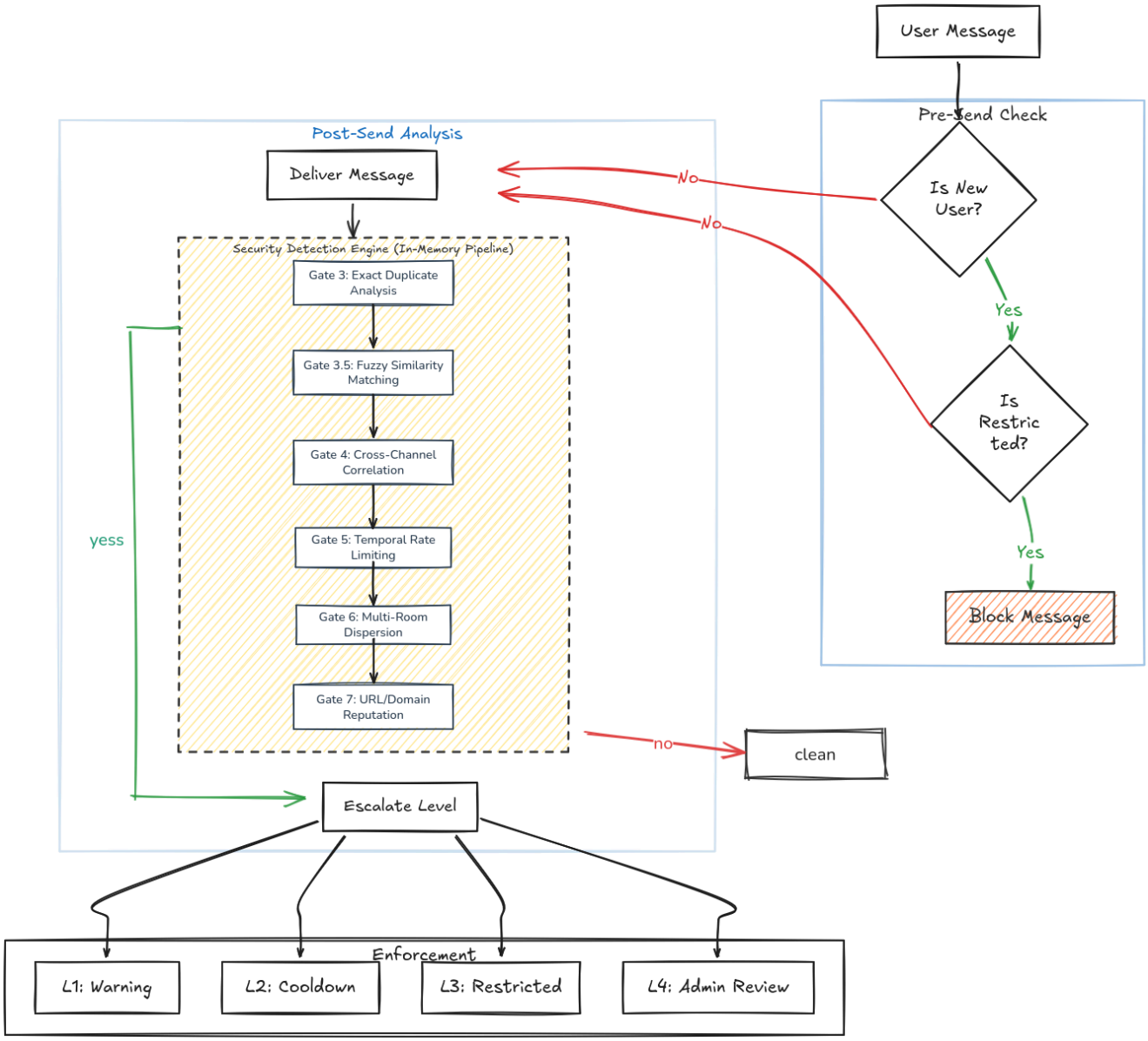
App workflow diagram



Implementation of Idea

The New Users Anti-Spam Monitoring System is structured around three core layers that work together: **the detection pipeline, the Moderation Level system,** and the **admin control layer**. Each layer has a distinct responsibility and together they take the system from raw message input to a fully explainable, human validated moderation decision.

Architecture diagram



Message Pipeline Integration

The first thing the app does is hook into the message pipeline through two entry points. The beforeSave handler runs before a message is written to the database. If a user is currently under an active restriction, the message is silently dropped here and never reaches storage or gets broadcast to the channel. The afterSave handler runs after a message is confirmed and is where the full detection pipeline executes. This two hook design keeps the hot path fast and ensures restricted users cannot simply retry and get through.

The Detection Pipeline

Detection runs as a sequence of gates. A message only moves to the next gate if it clears the current one, which keeps unnecessary computation off the critical path. All detection happens in memory against an in-memory cache scoped per user, so 99% of decisions never touch persistence at all.

Gate 3 is exact duplicate detection. Every incoming message is normalized and hashed, stripping invisible unicode, emojis, punctuation, and URLs before hashing so the comparison is always against the semantic content of the message. The hash is checked against the in-memory cache for the sending user within the configurable sliding time window. A match flags the user immediately.

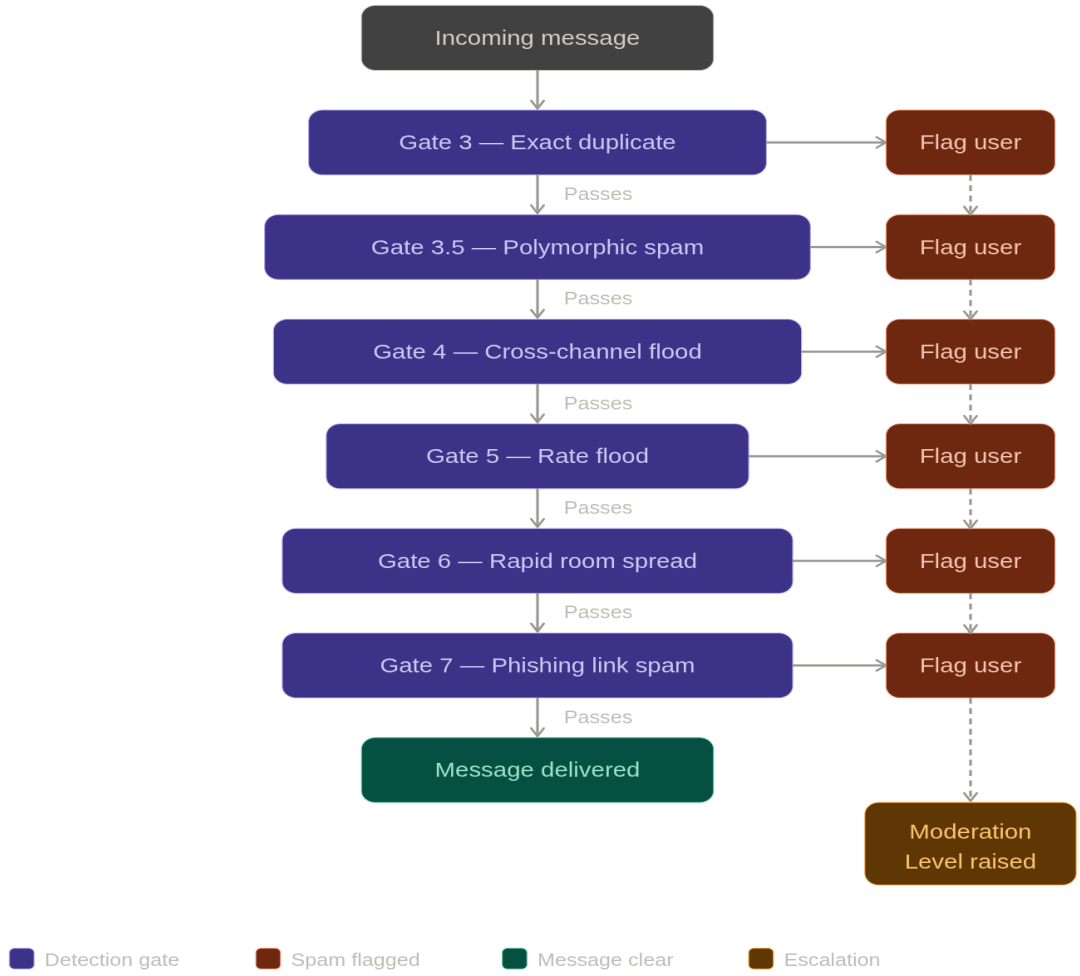
Gate 3.5 handles polymorphic spam. Real world spammers rarely send byte perfect duplicates. They add emojis, change capitalisation, strip punctuation, or pad with spaces to bypass hash checks. Gate 3.5 runs cosine similarity on tokenized word frequency maps, comparing the incoming message against recent messages from the same user. The similarity threshold is adaptive based on message length, stricter for short messages and more lenient for longer ones, to avoid false positives on brief conversational replies.

Gate 4 is cross channel exact flood detection. If the same hash appears across three or more channels within the sliding window it is flagged as a coordinated spam campaign regardless of how the content reads in isolation.

Gate 5 is rate flood detection. If a user sends multiple messages more than the set limit within 30 seconds or twelve or more within two minutes regardless of content, the velocity itself is the signal. This threshold is configurable from the admin settings panel so workspace admins can tune it based on their community size and activity level.

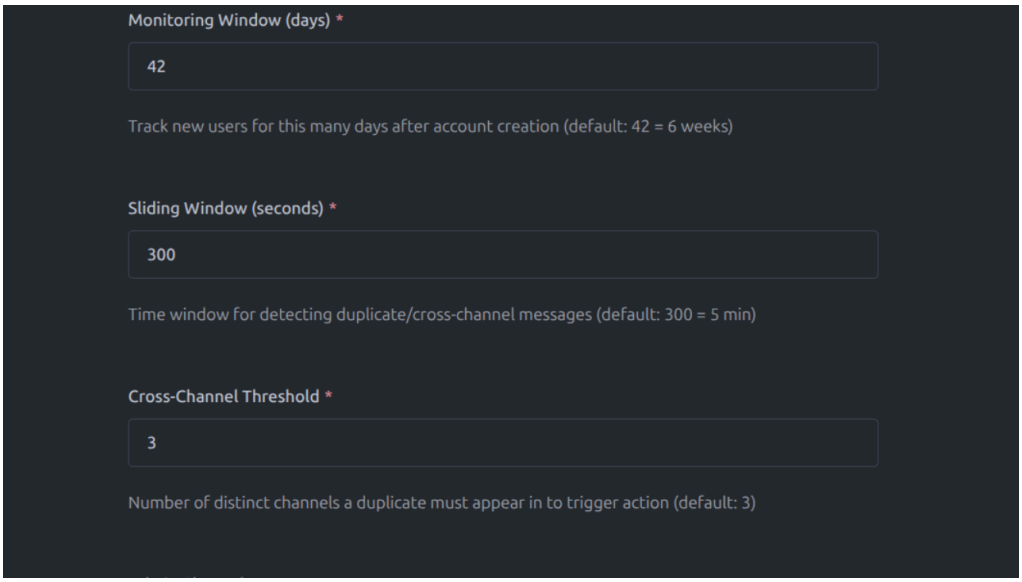
Gate 6 is rapid room spread detection. A user jumping across three or more distinct channels within two minutes while maintaining a high message rate is a strong indicator of spray and pray spam even when individual messages look clean.

Gate 7 is phishing link spam detection. If a user sends three or more URL bearing messages across two or more rooms within two minutes the system flags it as likely phishing or promotional spam regardless of the surrounding text.



detection flow chat

The **sliding window duration** and the **cross channel threshold** are both configurable from the admin settings panel, which means workspace admins can tune sensitivity to match their community size and tolerance.



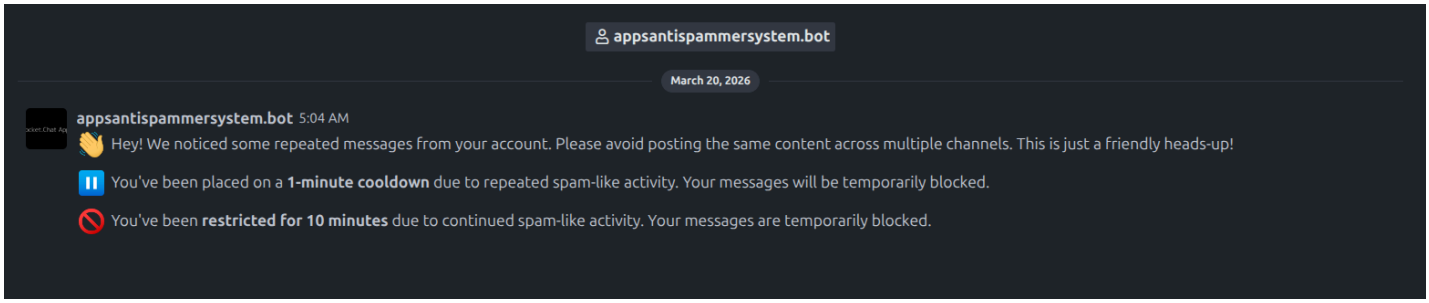
Moderation Level System

When a user is flagged the system does not jump straight to a ban. It responds through the Moderation Level system, a four tier graduated response where each level is proportional to how much suspicious behavior has accumulated.

- **Level 1** is a friendly DM sent to the user letting them know their behavior has been noticed.
- **Level 2** introduces a one minute posting cooldown.
- **Level 3** escalates to a ten minute restriction.
- **Level 4** is a permanent block that halts all posting until an admin explicitly reviews and makes a decision.

Levels persist across cooldowns so repeat offenders escalate naturally and a user cannot simply wait out a cooldown and start fresh. Only an admin vouch resets a user back to zero.

The system also notifies the user at each escalation so they are never silently restricted without context. A bot message is sent directly to the flagged user explaining their current status and what triggered it.

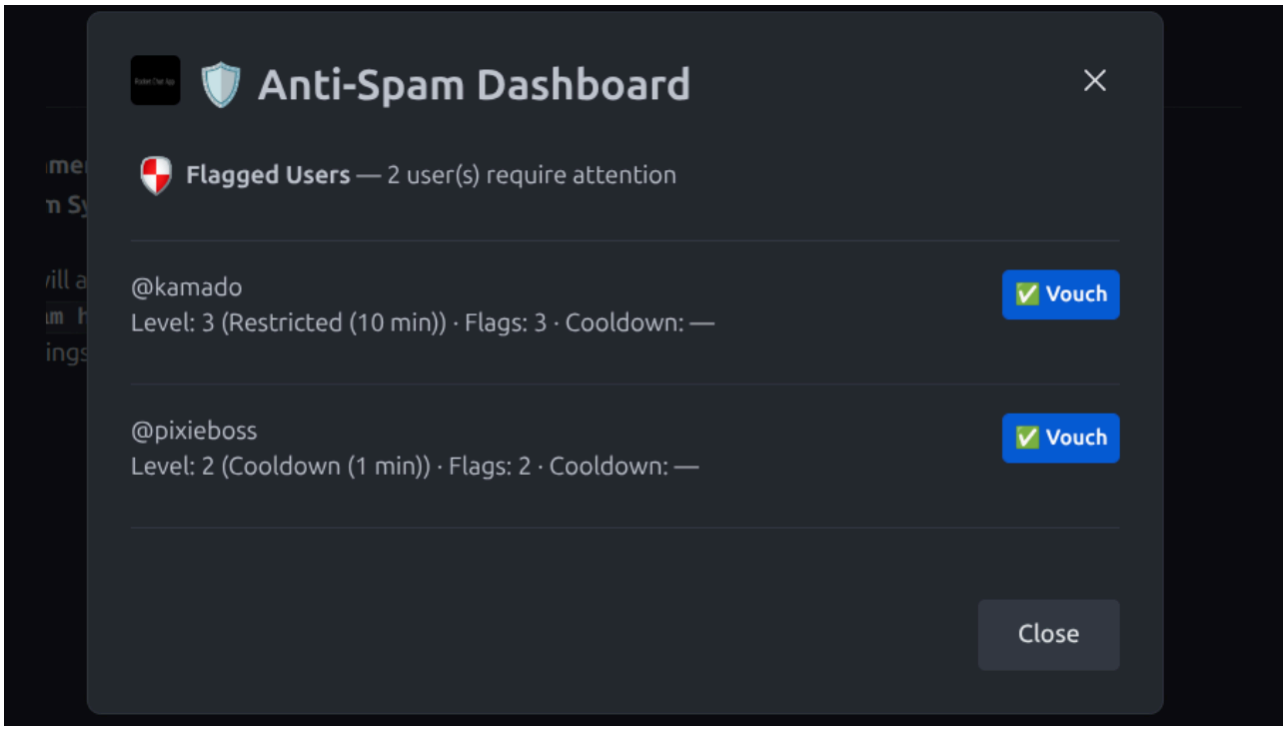


Admin Control Layer

The admin layer is where everything becomes visible and actionable. On install the app creates a private admin channel that serves as the nerve center for all moderation activity. Every flag, every escalation, and every restriction generates a notification in this channel with full context attached.

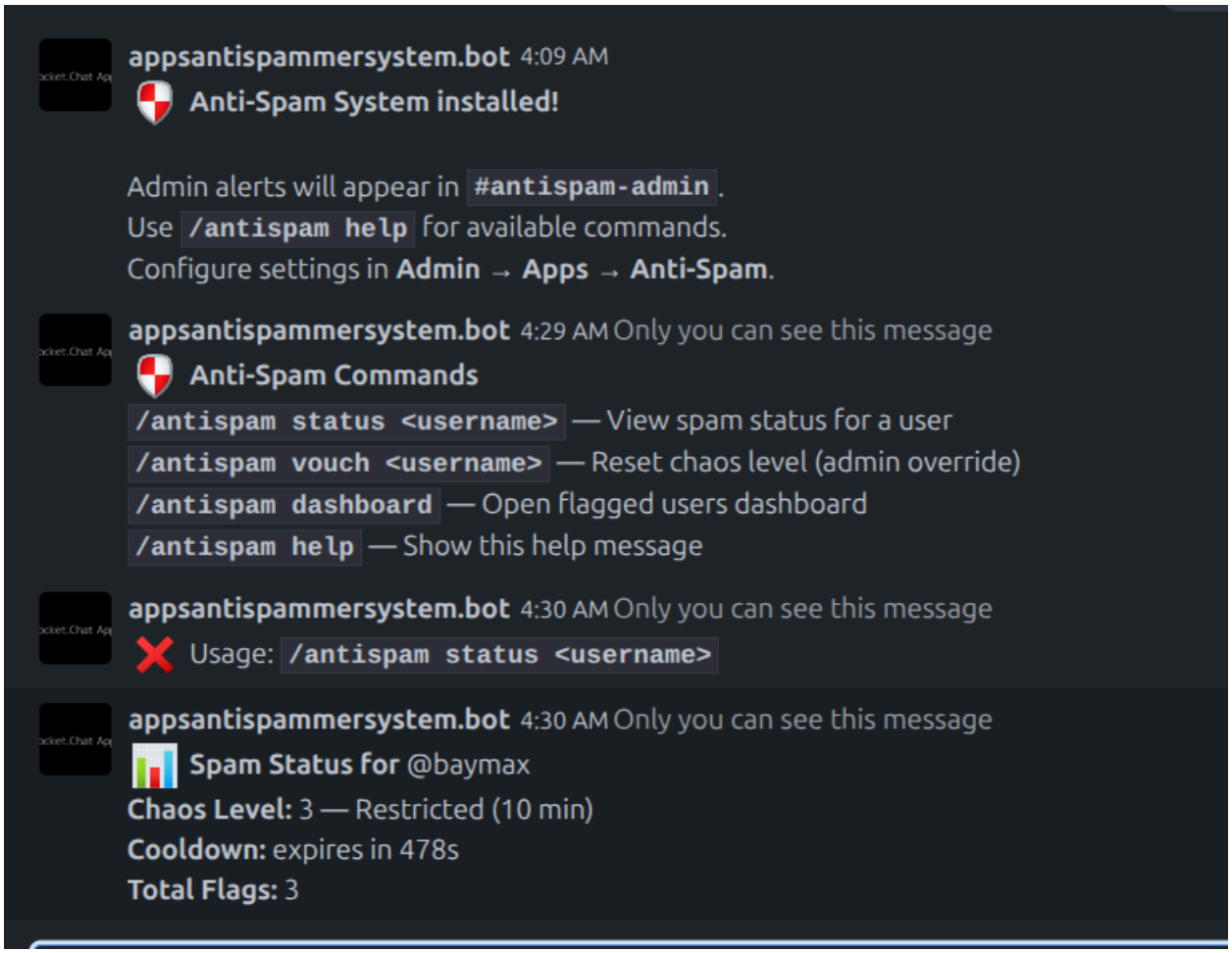
Notifications include a curated history of the flagged user's recent messages so the admin is never making a decision blind. Alongside the message history the notification surfaces the detection reason, the current Moderation Level, and the flag count, giving the admin everything they need to make a confident call.

The admin dashboard gives a live view of all currently flagged users, their Moderation Level, flag count, and cooldown status. From here admins can vouch for a user which immediately clears their record and resets them to level zero, or confirms a block if the behavior warrants it.



Dashboard for admin to see the flagged spammers

Admins can also interact with the system directly through slash commands inside the admin channel. Commands like /antispam status let admins query the current state of any user without opening the dashboard, keeping common actions fast and accessible from anywhere in the workspace.



What makes this system genuinely strong is how deliberately its three layers work together. The detection pipeline catches spam fast, before it spreads across channels and disrupts real conversations. The Moderation Level system ensures every response is proportional and escalates only when behavior actually warrants it. And the admin control layer gives the people running these communities complete visibility and the final say on every meaningful decision. Each layer is built to complement the others, and together they form a moderation system that is proactive, transparent, and firmly in the hands of the admins who know their communities best.

Known Limitations

Building this as a Rocket.Chat App on the Apps Engine comes with a set of architectural constraints that are worth being transparent about.

The in-memory cache that powers the hot path detection does not sync across instances in a multi-instance deployment. In a clustered environment a spammer could theoretically hit different server instances and slip past the cache based gates before the flag is written to persistence. The Moderation Level and flag count are always consistent across instances since they live in the Apps Engine persistence layer, so the spammer will still be caught, but there is a window where one or two messages could get through before the system catches up. For the vast majority of open community RC workspaces running on a single instance this is not a concern. For production multi-instance deployments the mitigation is shifting more detection state to persistence and **accepting slightly higher latency on the hot path**, which the current architecture supports as a clean upgrade path without requiring a rewrite.

The Apps Engine also hooks into the message pipeline at the app level, which means messages sent directly through the RC REST API can bypass the beforeSave and afterSave hooks entirely. A sufficiently sophisticated spammer who knows the API could exploit this to post without triggering detection. This is a known limitation of any app based approach rather than a specific weakness of this implementation, and it applies equally to any moderation system built outside of RC core.

Both of these are documented here not as blockers but as honest boundaries of what the app based approach can. The system is designed to handle the overwhelming majority of real world spam scenarios effectively while remaining transparent about where its boundaries are.

Detailed Work Plan

Application Review Period

During the application review phase, I will focus on deepening my understanding of the Rocket.Chat Apps Engine and its TypeScript definitions, particularly around the message pipeline hooks and persistence interfaces that are central to this project. I will study existing Rocket.Chat Apps to understand how production level implementations handle edge cases and performance at scale. Alongside this, I will continue refining the POC we have already built, stress testing the detection pipeline against a wider variety of spam patterns and tightening the Moderation Level logic based on what we observe. I will also actively engage with my mentors and the broader contributor community to ensure the foundation I bring into the coding period is as strong as possible.

Community Bonding Period

During the community bonding period, my primary goal is to establish a regular meeting cadence with my mentors Gabriel and Alfredo to align on project milestones, discuss implementation priorities, and agree on a detailed work schedule for the coding phase. I will draft a comprehensive plan covering each deliverable and share it early for feedback so there are no surprises once coding begins.

Beyond my mentors, I will reach out to experienced Rocket.Chat app developers and GSoC alumni to learn from their experiences and anticipate challenges before they become blockers. Having already been actively involved in the Rocket.Chat contributor community through weekly meetings and ongoing engagement, I intend to use this period to deepen those relationships further and stay informed on any new developments in the Apps Engine that could affect our implementation.

Having already contributed to the Rocket.Chat community through active participation in weekly meetings and ongoing engagement with contributors, I intend to use this period to broaden those interactions further, gaining a deeper understanding of the ecosystem and staying informed on new developments in the Apps Engine, UIKit, and core repository.

Detailed Breakdown of each week with specific focus points according to the standard 8-week project :

Week 1: Detection Pipeline Finalization and Settings Panel-

The core detection pipeline covering all seven gates is already built and tested as part of the POC. Week 1 will focus on finalizing the admin settings panel that exposes all configurable thresholds including sliding window duration, cross channel flood threshold, rate flood limits, new user monitoring window, and Moderation Level escalation thresholds. Alongside this, a UX flow document will be produced covering how the admin panel, dashboard, and notification system work together, addressing scale considerations for workspaces with large numbers of flagged users.

Week 2: Behavioral Scoring Engine Core Signals -

Implement the behavioral scoring engine that tracks posting frequency, link density, channel diversity, and message timing patterns across the full monitoring window. Define how individual signals are weighted and combined into a per user risk score that persists and updates with each new message.

Week 3: Behavioral Scoring Engine Integration and Tuning -

Integrate the scoring engine with the existing detection pipeline so risk scores feed directly into Moderation Level decisions. Test scoring behavior across a range of simulated user patterns including legitimate new users, borderline cases, and clear spammers. Tune signal weights based on observed outcomes and document the scoring model.

Week 4: Enriched Admin Notifications -

Extend the admin channel notification system to include a curated message history for every flagged user alongside their current risk score, Moderation Level, flag count, and the specific detection reason that triggered the alert.

Admins should have everything they need to make a confident decision directly from the notification without opening the dashboard.

Week 5: AI Powered User Summary and Risk Insights -

Integrate an AI layer that allows admins to query any flagged user directly from the admin channel and receive an intelligent summary of their activity. Given a user's message history, flag reasons, risk score progression, and behavioral signals, the AI synthesizes a clear human readable assessment of what the user did, how the pattern developed over time, and what the likely intent is. This gives admins genuine context when making vouch or block decisions rather than having to piece together raw data themselves.

Week 6: Admin Dashboard Enhancements and Scale Handling -

Extend the admin dashboard with full Moderation Level customization, allowing admins to define thresholds, cooldown durations, and escalation behavior per level directly from the UI. Implement pagination and filtering by Moderation Level and flag count so the dashboard remains performant and usable as the number of flagged users grows. Ensure vouch and block actions work cleanly from both the dashboard and admin channel notifications.

Week 7: End to End Integration Testing -

Run the full system end to end across a wide range of scenarios including exact duplicate spam, polymorphic variations, cross channel floods, legitimate new users, and edge cases around the monitoring window boundary. Document results, identify any gaps in detection accuracy or scoring behavior, and address them before the final week.

Week 8: Documentation, Polish, and Submission -

Write complete documentation covering installation, configuration, the full detection pipeline, known limitations including the multi-instance cache sync tradeoff, and guidance for future maintainers. Conduct a final round of testing

across all features, address any remaining issues from week seven, and prepare the project for submission and community handoff.

Future Work / After GSoC Period

After GSoC concludes, I intend to keep actively developing and maintaining the app with the guidance of my mentors until it reaches a stable release. I will engage with real workspace admins who deploy the system, gather feedback from actual moderation scenarios, and use those insights to refine detection thresholds, scoring weights, and the overall admin experience. I am committed to keeping the app aligned with any changes in the Rocket.Chat Apps Engine and will remain open to extending it with new capabilities suggested by the community. The goal is to see this land on the Rocket.Chat Marketplace as a genuinely useful, production ready moderation tool that open communities can rely on.

Shipping the Product

My goal is to see this system become a standard part of how open Rocket.Chat communities handle trust and safety, publicly available on the Marketplace and actively maintained. My ultimate aspiration is for any workspace administrator to be able to install it, tune it to their community's needs through a clean settings panel, and have a proactive moderation layer running within minutes, without needing to write a single line of code or understand anything about how the detection works under the hood.

Rocket.Chat Community

Over the past six months I have been actively involved in the Rocket.Chat ecosystem as a contributor across multiple projects. I have contributed to the Quick Replies App, Embedded Chat, AIAutomatedWorkflows, and the core Rocket.Chat repository, gaining hands-on experience with different parts of the platform and its development culture.

Through this work I have built a solid understanding of the Apps Engine, UIKit, and how real world Rocket.Chat Apps are structured and maintained. These contributions have also given me direct insight into the limitations and pain points that developers face when building on the platform, which is a big part of what motivated this project.

After GSoC I look forward to continuing my involvement across the Rocket.Chat ecosystem and contributing to more projects as the community grows.

Past work done

Over the past six months I have been actively contributing across multiple Rocket.Chat projects, building hands-on experience with the Apps Engine, core composer, UIKit, and AI integrations.

RocketChat/Apps.QuickReplies

- Implemented /quick grammar command that fixes grammatical and spelling errors in messages before sending, powered by the existing AI handler pattern while preserving original meaning. Also updated the outdated Gemini model to the latest API version, restoring full AI functionality for all users : link
- Fixed a critical null check bug in contextual bar overflow interactions where room was returned as null instead of undefined, causing silent failure when opening modals from the contextual bar:link

RocketChat/Embeddedchat

- Added smart Markdown list formatting with auto-continuation to the message composer, matching the behavior of modern editors like Slack and Notion. Supports auto-incrementing numbered lists, auto-continuing bullet lists, and exiting list mode on empty items via Shift+Enter : link
- Fixed a silent failure where UIKit contextual bars triggered from slash commands were not rendering in EmbeddedChat. The contextual bar now opens fully, renders correctly, and all interactive elements including buttons, inputs, and dropdowns work as expected. : link

- Added a Keyboard Shortcuts entry to the EmbeddedChat three dot options menu that opens a reference modal listing all supported shortcuts, improving discoverability for users. : link

RocketChat/Rocket.Chat

Added ordered and unordered list formatting buttons to the core Rocket.Chat message composer toolbar, bringing the same smart list continuation behavior to the main platform. Currently in review with the frontend team. : link

RocketChat/AutomatedChatWorkflows

- Replaced multiple individual slash commands with a unified workflow contextual bar UI, significantly improving the user experience for managing chat automations. Users can now view, edit, enable, disable, and delete workflows from a single interface with overflow menus and dedicated modals: link
- Fixed an AI response parsing issue by correctly reading through content instead of data, and added descriptive error handling for missing choices in the AI response: link

Work Done Related to This Project

I have built a complete working proof of concept for this project. The full repository is available here: repository
Demo of the proof of concept : Demo

- Implemented polymorphic spam detection using cosine similarity on token frequency maps, tested against real variations sent simultaneously across channels and caught correctly. Demo in readme , demo showed in RC channel

- The admin dashboard is live with flagged user tracking, Moderation Level status, and vouch functionality. Admins can also interact directly through slash commands from the admin channel. Demo in readme , demo showed in RC channel

- When a user triggers detection, the bot pings them directly with a warning or timeout notice based on their current Moderation Level. Messages from restricted users are silently dropped before ever reaching the database. Demo in readme , demo showed in RC channel

Why me?

My interest in this project did not start with this proposal. It started with a real problem I noticed in open communities and a genuine curiosity about how to solve it properly. That curiosity led me to build the proof of concept you have seen throughout this proposal, shaped by mentor feedback, community discussions, and continuous iteration rather than built in isolation.

Across my projects I have worked extensively with security and privacy sensitive systems, from implementing content hashing and data integrity layers to building ERP systems where moderation, access control, and audit trails are not optional features but core requirements. That experience gives me a grounded understanding of what it means to build systems where decisions need to be explainable, reversible, and trustworthy, which is exactly the philosophy this project is built on.

I have also been working as an SDE intern for the past three months, which has helped me develop habits around writing maintainable, production quality code that others can build on.

Within the Rocket.Chat community I have stayed consistently engaged, attending gatherings, contributing across multiple projects, and actively seeking feedback from mentors and fellow contributors. I started from a rough idea, went through real architectural discussions, and arrived at the current design through genuine conversation with the community. I am not claiming to have all the answers. What I can say is that I care deeply about seeing this become something genuinely useful for the Rocket.Chat ecosystem and I am committed to putting in the work to make that happen.

Time Availability

As a Google Summer of Code participant, I am fully committed to dedicating the time and effort required to deliver this project successfully within the 175 hour period. I will maintain consistent daily communication with my mentors, sharing progress updates and raising any blockers early so they can be addressed without impacting the timeline. I am ready to put in whatever it takes to see this project through to a high quality, complete outcome.